Akash Banerjee

Contact

akashb.me
akash@akashb.me
github.com/TIFitis
LinkedIn

United Kingdom

+44 7471 050392

Core Stack

C, C++
LLVM, MLIR, Flang
OpenMP offload
OpenMPIRBuilder
AMDGPU offload
Linux
Python

Engineering

Compiler optimisation
Low-level debugging
Program analysis
Regression testing
CI/CD
Git, GDB, LLDB

Languages

English
Hindi
Bengali

Education

M.Tech., CSE
IIT Hyderabad
9.50/10, 2021

B.Tech., CSE
RERF, Kolkata
8.37/10, 2017

Profile

Staff compiler engineer specialising in OpenMP target offload for LLVM Flang and MLIR. I work across Fortran lowering, the OpenMP dialect, FIR-to-LLVM conversion, LLVM IR translation and OpenMPIRBuilder, focusing on data mapping, target-data constructs and shared AMDGPU offload code generation. My background includes compiler optimisation, program analysis and formal verification.

Experience

2026–Pres **Member of Technical Staff** — AMD, UK

2022–2026 **Senior Compiler Engineer** — AMD, UK

Across both AMD roles, I develop OpenMP target offload for LLVM Flang and MLIR across Fortran lowering, dialect design, FIR/LLVM conversion, LLVM IR translation and OpenMPIRBuilder. My work combines language and IR design with production debugging and upstream maintenance. Recognition includes multiple **AMD Spotlight Awards** and an **AMD Executive Award**.

2021–2022 **Graphics Compiler Engineer** — Imagination Technologies, UK

Developed and debugged code-generation paths in a proprietary C++ graphics compiler backend, focusing on generated-code correctness and performance.

Key Compiler Projects

2023–Pres **OpenMP data mapping and declare mappers** — LLVM upstream

Built major parts of MLIR's OpenMP mapping path: map representation and target-region mapping; `declare mapper` operations and lowering; map-clause attachment; FIR conversion and LLVM IR translation; and default mapping for Fortran derived types and allocatable components.

2026 **OpenMP mapping semantics and diagnostics** — LLVM upstream

Fixed mapper attachment and recursive emission for partial maps and nested types, tightened implicit-mapper rules for pointer captures and data-motion directives, and added Flang diagnostics for unsafe descriptor mappings.

2025 **Inferred and derived-type mapping** — LLVM upstream

Implemented AUTOMAP lowering and an FIR pass that converts inferred mappings into explicit target-data operations; extended implicit mapping to nested allocatable components of Fortran derived types.

2025 **AMDGPU complex-type and math lowering** — LLVM upstream

Added AMDGPU handling for complex function arguments and results, together with complex operations and ROCDL lowerings required by Flang workloads.

2022–2024 **Shared OpenMP offload code generation** — LLVM upstream

Moved reusable offload paths from Clang into OpenMPIRBuilder: mapping arrays and descriptors, target-data calls, device privatisation, dispatch utilities, GPU reductions and mapper emission. Clang and MLIR reuse these paths.

2023 **OpenMP target-data constructs in Flang/MLIR** — LLVM upstream

Defined MLIR operations for `target data`, `target enter data` and `target exit data`, then added Flang lowering, FIR-to-LLVM conversion, OpenMPIRBuilder translation and map support for target regions.

Publication

Nov. 2025 **Implementing OpenMP Offload Support in the AMD Next Generation Fortran Compiler**

D. Adamski, S. Afonso, **A. Banerjee**, et al. In *SC Workshops '25*, pp. 1028–1038. Describes the architecture and implementation of OpenMP offload in LLVM Flang. ACM DOI.